

language: es

Característica: Procesamiento de mensajes de regrabación de tarjetas

Como sistema tarjetas-service

Quiero consumir mensajes de la cola tarjetas-regrabacion-creada

Para registrar las tarjetas regrabadas y actualizar los datos del cliente

Antecedentes:

Dado que el sistema tarjetas-service está configurado y en ejecución

Y la conexión con RabbitMQ está establecida

Y la cola "tarjetas-regrabacion-creada" existe y está disponible

CASO DE USO 1: Registro exitoso de nueva tarjeta

Escenario: Procesar mensaje de regrabación para tarjeta nueva

Dado que no existe un registro activo para la tarjeta "4794230592497775"

Cuando llega el siguiente mensaje a la cola:

""

```
{
  "codPersona": "23546568",
  "fechaRespuesta": "2026-03-06",
  "tipoTarjeta": "TD",
  "numeroTarjeta": "4794230592497775",
  "nombrePlasticoTarjeta": "JOSÉ ORTIZ",
  "fechaEmisionTarjeta": "06/03/26",
  "fechaVencimientoTarjeta": "202903",
  "afinidad": {
    "codigo": "500",
    "descripcion": "VISA PREPAGA"
  },
  "marca": {
    "codigo": "V",
    "descripcion": "VISA"
  }
}
```

PROF

Entonces el sistema debe insertar un nuevo registro en la tabla "tarjetas_regrabacion"

Y el registro debe tener estado "ACTIVO"

Y debe publicarse un evento interno "TarjetaRegrabadaEvent"

Y el evento interno debe ejecutarse en una transacción independiente

Y debe confirmarse el ACK del mensaje a RabbitMQ

Y debe registrarse la auditoría del INSERT

CASO DE USO 2: Detección y manejo de duplicado activo

Escenario: Detectar duplicidad de tarjeta activa existente
Dado que existe un registro activo para la tarjeta "4794230592497775"

campo	valor	
cod_persona	23546568	
tipo_tarjeta	TD	
numero_tarjeta	4794230592497775	
estado	ACTIVO	

Cuando llega un mensaje con el mismo número de tarjeta "4794230592497775"
Entonces el sistema debe detectar la duplicidad
Y no debe insertar un nuevo registro
Y debe registrar un log de advertencia "Registro duplicado detectado"
Y debe confirmar el ACK del mensaje (descartar silenciosamente)
Y no debe publicar el evento interno de actualización

CASO DE USO 3: Reactivación de tarjeta previamente inactiva

Escenario: Reactivar tarjeta inactiva con nuevos datos
Dado que existe un registro inactivo para la tarjeta "4794230592497775"

campo	valor	
cod_persona	99999999	
tipo_tarjeta	TC	
numero_tarjeta	4794230592497775	
estado	INACTIVO	

Cuando llega un mensaje para la misma tarjeta con datos actualizados:

campo	valor nuevo	
codPersona	23546568	
tipoTarjeta	TD	

Entonces el sistema debe actualizar el registro existente
Y debe cambiar el estado a "ACTIVO"
Y debe actualizar los campos con los nuevos valores
Y debe publicar el evento interno de actualización
Y debe confirmar el ACK del mensaje

CASO DE USO 4: Validación de estructura de mensaje

Escenario: Rechazar mensaje con estructura inválida
Dado que llega un mensaje con formato JSON inválido
Cuando se intenta deserializar el payload
Entonces el sistema debe rechazar el mensaje
Y debe enviar el mensaje a la Dead Letter Queue (DLQ)

Y debe registrar el error de parsing en los logs

Y no debe intentar insertar en base de datos

CASO DE USO 5: Validación de campos obligatorios

Esquema del escenario: Validar campos obligatorios faltantes

Dado que llega un mensaje sin el campo "<campo_faltante>"

Cuando se procesa el mensaje

Entonces debe rechazarse el mensaje

Y debe registrarse el error "Campo obligatorio faltante: <campo_faltante>"

Y debe enviarse a DLQ

Ejemplos:

campo_faltante	
codPersona	
tipoTarjeta	
numeroTarjeta	
fechaVencimientoTarjeta	
afinidad	
marca	

CASO DE USO 6: Manejo de tipos de tarjeta

Escenario: Procesar diferentes tipos de tarjeta

Dado que llega un mensaje con tipoTarjeta "<tipo_tarjeta>"

Cuando se procesa el mensaje

Entonces el sistema debe aceptar el mensaje

Y debe registrar el tipo de tarjeta correctamente

Ejemplos:

tipo_tarjeta	descripcion	
TD	Tarjeta de Débito	
TC	Tarjeta de Crédito	

PROF

CASO DE USO 7: Transacciones independientes

Escenario: Aislamiento de transacciones T1 y T2

Dado que se inserta exitosamente un registro en T1

Cuando falla la transacción T2 (actualización cliente)

Entonces el registro en T1 debe persistir (COMMIT)

Y la transacción T2 debe hacer ROLLBACK

Y debe registrarse el error de T2

Y el sistema debe estar disponible para continuar procesando

CASO DE USO 8: Manejo de errores de base de datos

Escenario: Error de conexión a base de datos durante inserción
Dado que la base de datos no está disponible
Cuando se intenta procesar un mensaje válido
Entonces debe hacerse ROLLBACK de la transacción T1
Y debe reintentarse el procesamiento (configurable)
Y después de <max_reintentos> intentos, enviar a DLQ
Y debe alertar al equipo de operaciones

CASO DE USO 9: Concurrencia - Race condition

Escenario: Manejar mensajes duplicados simultáneos
Dado que dos mensajes idénticos llegan simultáneamente
Cuando ambos hilos intentan insertar el mismo número de tarjeta
Entonces solo uno debe tener éxito
Y el segundo debe detectar el duplicado tras el commit del primero
Y ambos mensajes deben confirmar ACK
Y debe haber un único registro ACTIVO en base de datos

CASO DE USO 10: Evento interno asíncrono

Escenario: Publicación asíncrona de evento interno
Dado que se completó exitosamente la inserción en T1
Cuando se publica el evento "TarjetaRegrabadaEvent"
Entonces la publicación debe ser asíncrona
Y no debe bloquear el hilo principal
Y el listener debe ejecutar en transacción T2 independiente
Y debe incluir todos los datos necesarios para actualización